

Class 9/17

About Team Projects

- **Two teams from my estimation is not making good progress**
 - We have 8 weeks roughly once we step into Oct
 - Still see some lack of coordination, e.g., two persons doing object detection
 - Too much saying what you are going to do, no testing or execution
 - No cohesiveness of how to combine the parts
- Speed and memory-efficiency (50%)
 - Credit will be given to project that can execute the models efficiently say on your phone.
- Creativity and novelty (20%)
 - Beating SOTA baselines
- Code clarity and correctness (20%)
- Clarity of report (10%)
 - Motivation of the idea is clearly articulated
 - Literature review is comprehensive
 - Experiments clearly showing good performance
 - A section on what was accomplished each week and the designation of tasks
- Outcome of the team project can be an interesting system or a publishable paper at CVPR or ICCV. For latter, there is potential to work with me further to develop it into a full paper or pass off to a PhD student.
- In addition, we will also look at the size of the project (on a scale of 0 to 1, where 1 is worthy of about 3.5 months of efforts from start of course to Dec 4th). This will be used as a multiplier - let's say you score 80 on the first three bullets but your project size is graded at 0.6 as it is not fully needing 3.5 months, then the final project score is 48.

Checkpoints

- Save your best checkpoint per assignment
- Ensure that you can load the datasets you used in your assignments onto your pop assignment environment
- I just realized that for the CLIP assignments, the HAR dataset has only 12k+ training samples, so the SGCLIP batch size maxed out at that, which is what you can use for batch size.

Transformer vs CNN

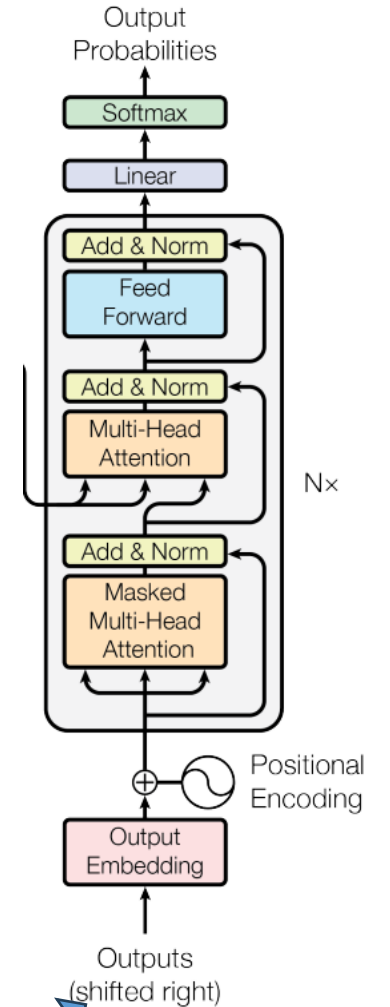
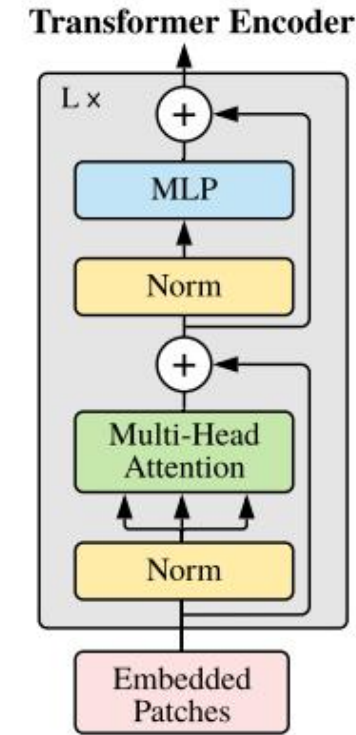
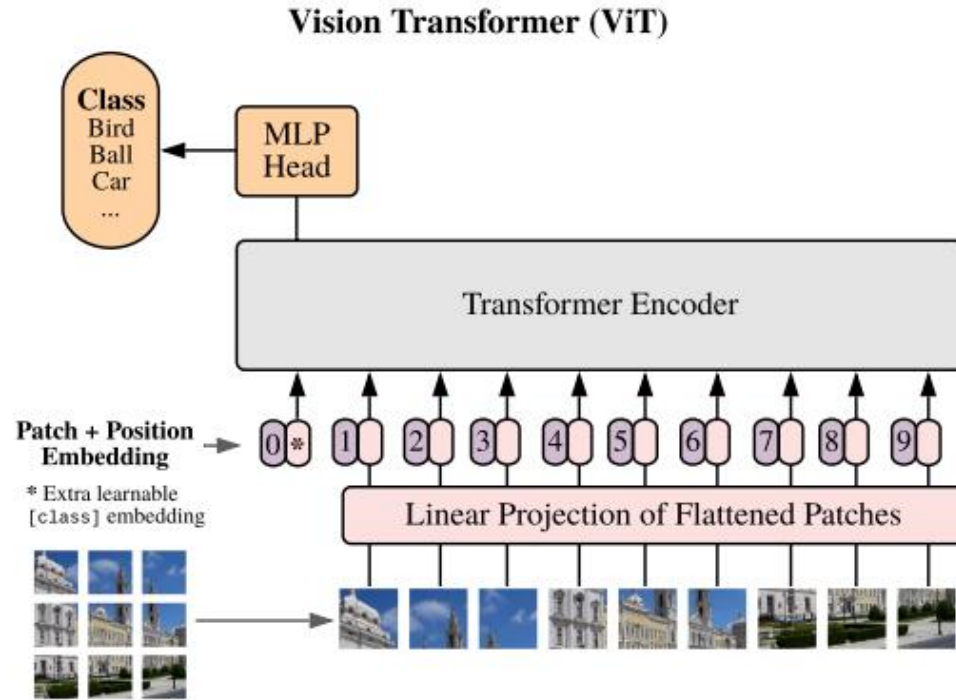
- Transformer:

- Originated from language research
- Sentences are broken into tokens
- Tokens are run through self-attention processor:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- Intuitively, and this is MY **interpretation**, this is like "self-selection" to figure out what tokens are important
- Convolutional layers are also trying to achieve similar purpose – selecting the neurons/pixels that matters, but done through a "completely" different mechanism (like abstracting an image)

ViT Architecture



$$\mathbf{z}_0 = [\mathbf{x}_{\text{class}}; \mathbf{x}_p^1 \mathbf{E}; \mathbf{x}_p^2 \mathbf{E}; \dots; \mathbf{x}_p^N \mathbf{E}] + \mathbf{E}_{\text{pos}},$$

$$\mathbf{z}'_\ell = \text{MSA}(\text{LN}(\mathbf{z}_{\ell-1})) + \mathbf{z}_{\ell-1},$$

$$\mathbf{z}_\ell = \text{MLP}(\text{LN}(\mathbf{z}'_\ell)) + \mathbf{z}'_\ell,$$

$$\mathbf{y} = \text{LN}(\mathbf{z}_L^0)$$

$$\mathbf{E} \in \mathbb{R}^{(P^2 \cdot C) \times D}, \mathbf{E}_{\text{pos}} \in \mathbb{R}^{(N+1) \times D}$$

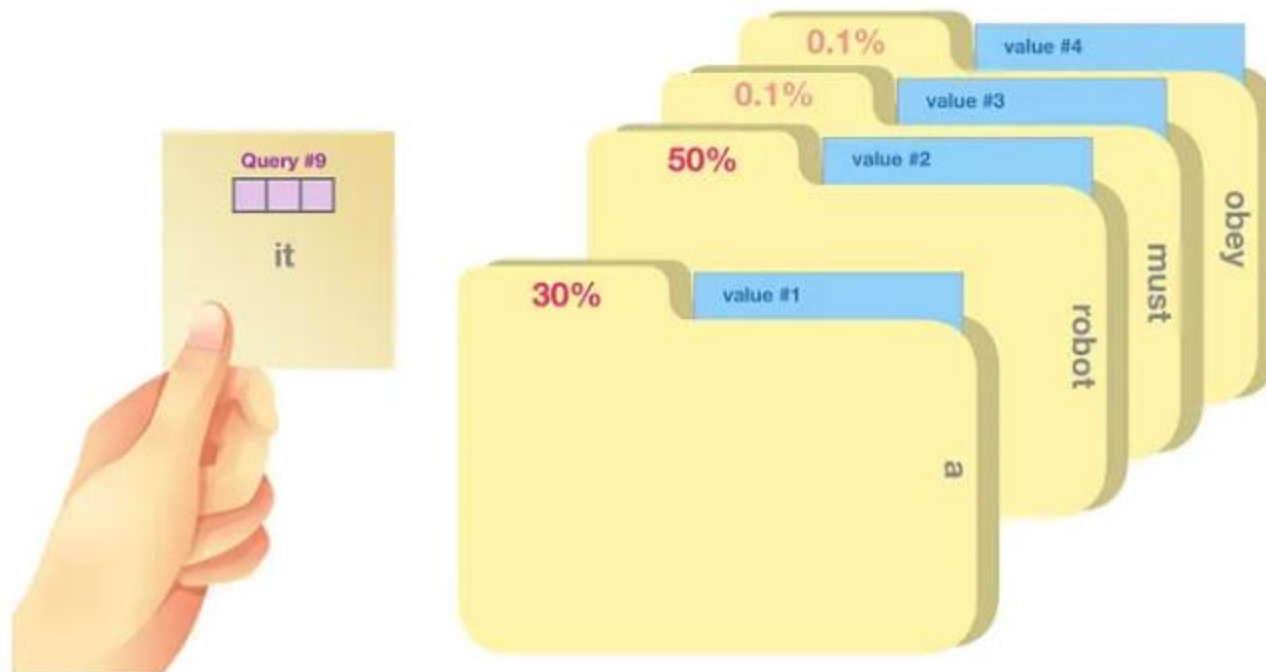
$$\ell = 1 \dots L$$

$$\ell = 1 \dots L$$

WHY?

QKV

Example on using LLaVA for
Classification



Positional Encoding

The Position Encoding is computed independently of the input sequence. These are fixed values that depend only on the max length of the sequence. For instance,

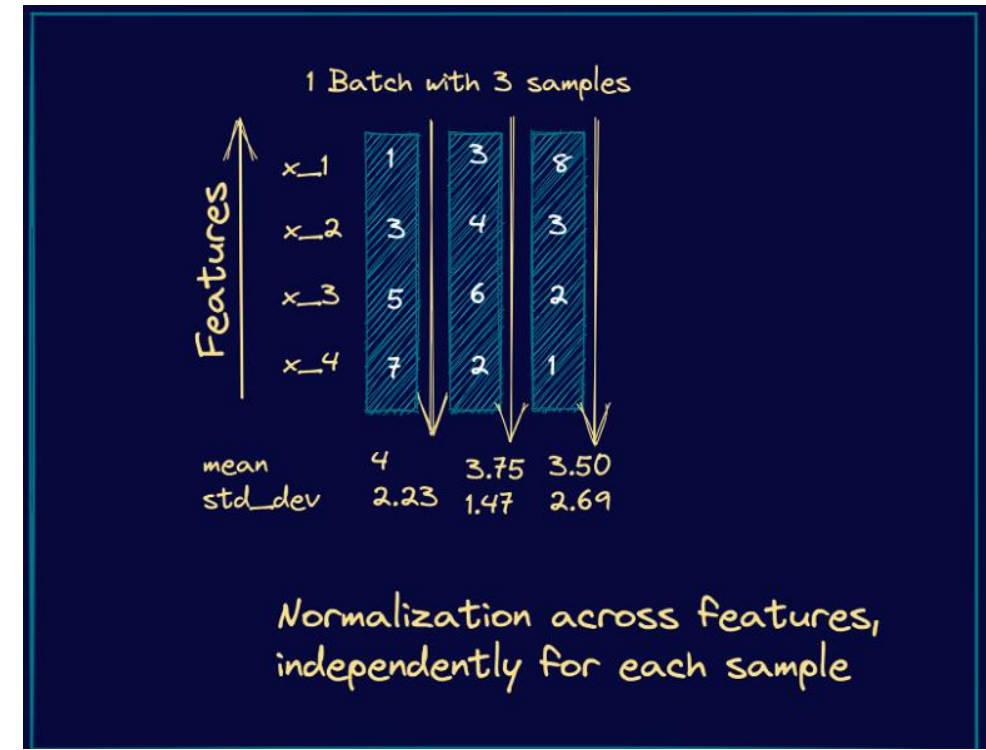
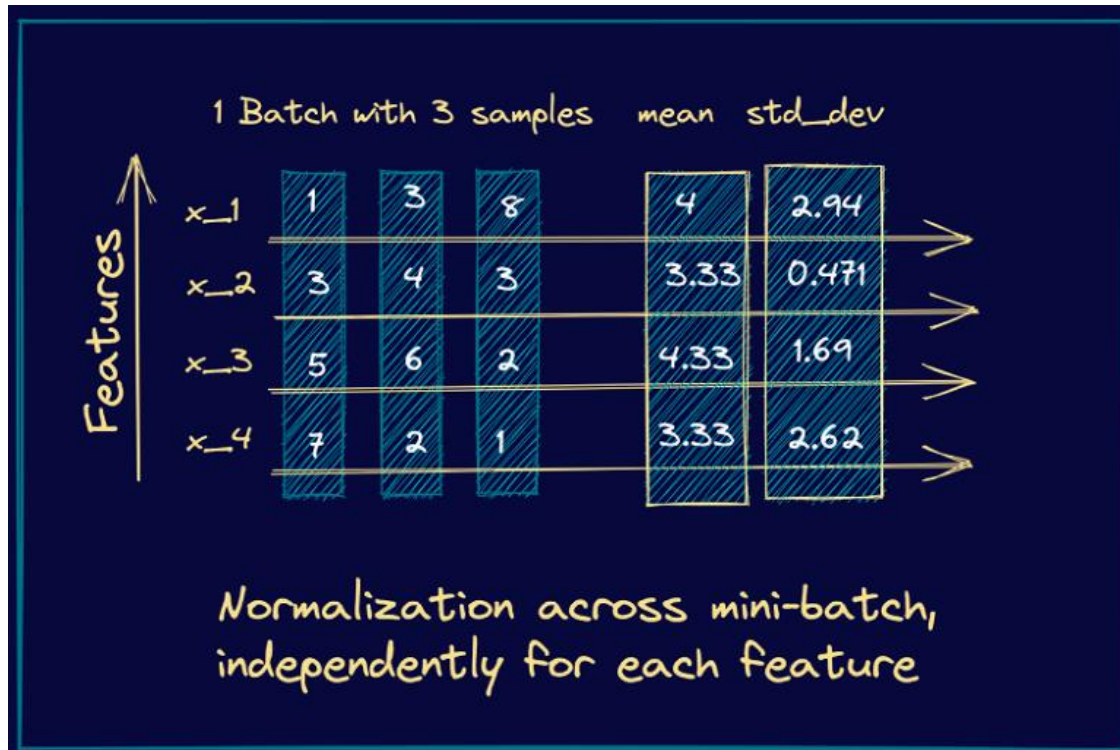
- the first item is a constant code that indicates the first position
- the second item is a constant code that indicates the second position,
- and so on.

These constants are computed using the formula below, where

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{model}})$$
$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{model}})$$

- pos is the position of the word in the sequence
- d_{model} is the length of the encoding vector (same as the embedding vector) and
- i is the index value into this vector.

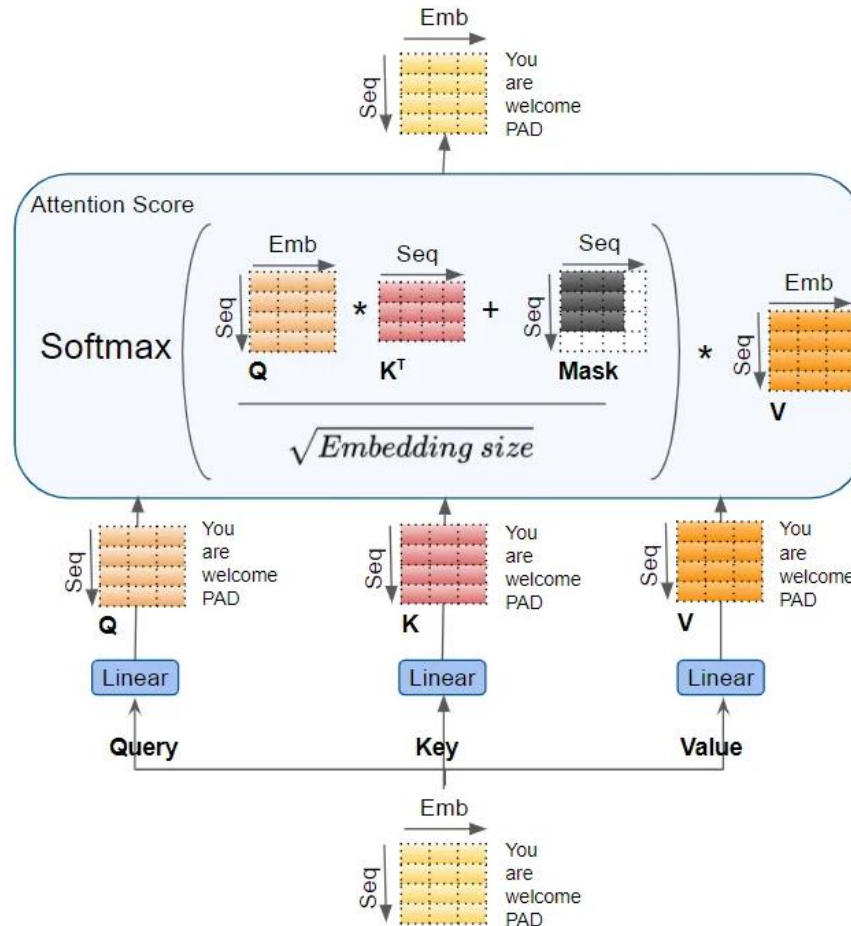
LN (Layer Norm) vs BN (Batch Norm)



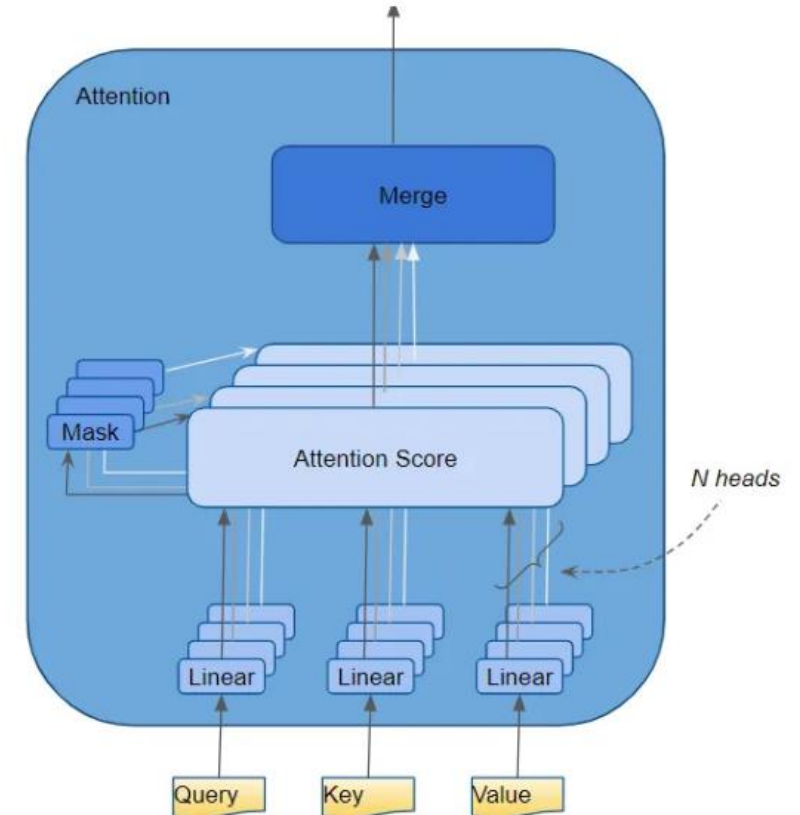
$$\begin{aligned}
\mathbf{z}_0 &= [\mathbf{x}_{\text{class}}; \mathbf{x}_p^1 \mathbf{E}; \mathbf{x}_p^2 \mathbf{E}; \dots; \mathbf{x}_p^N \mathbf{E}] + \mathbf{E}_{\text{pos}}, & \mathbf{E} &\in \mathbb{R}^{(P^2 \cdot C) \times D}, \mathbf{E}_{\text{pos}} \in \mathbb{R}^{(N+1) \times D} \\
\mathbf{z}'_\ell &= \text{MSA}(\text{LN}(\mathbf{z}_{\ell-1})) + \mathbf{z}_{\ell-1}, & \ell &= 1 \dots L \\
\mathbf{z}_\ell &= \text{MLP}(\text{LN}(\mathbf{z}'_\ell)) + \mathbf{z}'_\ell, & \ell &= 1 \dots L \\
\mathbf{y} &= \text{LN}(\mathbf{z}_L^0)
\end{aligned}$$

MSA (Multi-head Self Attention)

The Query, Key, and Value are each passed through separate Linear layers, each with their own weights, producing three results called Q, K, and V respectively. These are then combined together using the Attention formula as shown below, to produce the Attention Score.



The Transformer calls each Attention processor an Attention Head and repeats it several times in parallel. This is known as Multi-head attention. It gives its Attention greater power of discrimination, by combining several similar Attention calculations.



Who Attend Who?

- Self-attention in the Encoder — the input sequence pays attention to itself
- Self-attention in the Decoder — the target sequence pays attention to itself
- Encoder-Decoder-attention in the Decoder — the target sequence pays attention to the input sequence

Attendance

ViT Variants

Model	Layers	Hidden size D	MLP size	Heads	Params
ViT-Base	12	768	3072	12	86M
ViT-Large	24	1024	4096	16	307M
ViT-Huge	32	1280	5120	16	632M

ViT-L/16 means the “Large” variant with 16×16 input patch size.

Assignment Due 10/3

- Dataset:
<https://www.kaggle.com/datasets/shuvoalok/cityscapes> (gtFine_trainvaltest.zip only)
 - Task is semantic segmentation
 - Train has label masks
 - Val has label masks
- Take ViT-S pretrained from HF or anywhere you like:
 - Switch out the classification head for a semantic segmentation task
 - Train on "Train"
 - Evaluate on "Val"
- Report, code, video
- Please ensure you keep the checkpoints!!

